

FlashAttention-2: 通过更好的并行和工作划分实现更快的注意

Tri Dao^{1,2}

¹Princeton University 计算机科学系²Stanford
University 计算机科学系 trid@cs.stanford.edu

2023年7月18日

摘要

在过去几年中，将 Transformer 模型扩展到更长的序列长度一直是一个主要问题，这有望提升语言建模和高分辨率图像理解中的性能，并解锁代码、音频和视频生成中的新应用。注意层是扩展到更长序列的主要瓶颈，因为其运行时间和内存会随序列长度呈二次增长。FlashAttention [5] 利用不对称的GPU内存层次结构，带来显著的内存节省（线性而非二次）和运行时间加速比（2-4× 相较于优化过的基线方法），且无近似。然而，FlashAttention 仍远不及优化过的 GEMM（通用矩阵乘法）运算那样快速，仅能达到理论最大FLOPs/s 的 25-40%。我们观察到，这种低效率源于GPU上不同线程块与warp（束）之间次优的工作划分，导致要么低占用率，要么不必要的共享内存读/写操作。我们提出了 FlashAttention-2，通过更好的工作划分来解决这些问题。具体来说，我们（1）微调算法以减少非矩阵乘法的FLOPs（2）将注意计算（即使对于单个头）跨不同线程块并行，以提高占用率，（3）在每个线程块内，在warp（束）之间分配工作，以减少通过共享内存的通信。这些改进相较于 FlashAttention 带来了约2× 加速比，在 A100 上达到理论最大FLOPs/s 的 50-73%，并接近 GEMM运算的效率。我们的实证结果表明，当端到端用于训练 GPT 风格模型时，FlashAttention-2 在每块 A100 GPU 上可达到最高 225 TFLOPs/s 的训练速度（72% 模型FLOPs利用率）。¹

1 引言

提升 Transformer 模型 [18] 的上下文长度的扩展性是一项挑战，因为其核心的注意层在运行时间和内存方面的需求与输入序列长度呈平方关系。理想情况下，我们希望突破标准的 2k 序列长度上限，以训练模型去理解书籍、高分辨率图像以及长时段视频。就在过去的一年里，已经出现了数个相比以往具有更长上下文的语言模型：上下文长度为 32k 的 GPT-4 [12]、上下文长度为 65k 的 MosaicML 的 MPT，以及上下文长度为 100k 的 Anthropic 的 Claude。诸如长文档查询和故事写作等新兴用例表明，人们确实需要具备如此长上下文的模型。

为降低在如此长上下文上进行注意的计算需求，已经提出了众多用于近似注意的方法 [2, 3, 4, 8, 9, 14, 19, 20]。尽管这些方法已有一些使用案例，据我们所知，大多数大规模训练运行仍然使用标准注意。受此启发，Dao 等人 [5] 提出通过重新排序注意计算，并利用经典技术（tiling、recomputation）来显著加速，并将内存使用从与序列长度成二次关系降低为线性。这带来了相对于优化的基线方法 2-4× 墙钟时间加速比，内存节省最高可达 10-20×，

¹可在 <https://github.com/Dao-AILab/flash-attention> 获取 FlashAttention-2

无需近似，因此 FlashAttention 已在 Transformer 模型的大规模训练和推理中得到广泛采用。

然而，即使上下文长度进一步增加，FlashAttention 仍然远不如诸如矩阵乘法（GEMM（通用矩阵乘法））等其他原语高效。具体而言，尽管 FlashAttention 已经比标准注意实现快 2-4 $\times$ 倍，其前向传播仅能达到设备理论最大 FLOPs/s 的 30-50%（图 5）；而反向传播更为具有挑战性，在 A100 GPU 上仅能达到最大吞吐量的 25-35%（图 6）。相比之下，优化的 GEMM（通用矩阵乘法）可以达到设备理论最大吞吐量的 80-90%。通过细致的性能分析，我们观察到 FlashAttention 在 GPU 上不同线程块与 warp（束）之间的工作划分仍不理想，导致要么占用率低，要么出现不必要的共享内存读取/写操作。

在 FlashAttention 的基础上，我们提出了具有更好并行和工作划分的 FlashAttention-2，以解决这些挑战。

1. 在第3.1节中，我们微调算法，以在不改变输出的情况下减少非矩阵乘法 FLOPs 的数量。尽管非矩阵乘法 FLOPs 只占总 FLOPs 的一小部分，但由于 GPU 具有用于矩阵乘法的专用单元，执行这些操作更耗时，因此矩阵乘法吞吐量最高可比非矩阵乘法吞吐量高出 16 $\times$ 。因此，减少非矩阵乘法 FLOPs，并尽可能多地进行矩阵乘法 FLOPs 十分重要。
2. 我们建议在批次和注意力头数维度之外，还沿着序列长度维度对前向传播和反向传播进行并行。这在序列较长（因此批大小往往较小）的情况下可以提高占用率（GPU 资源利用率）。
3. 即使在一个注意计算块内，我们也在一个线程块的不同 warp（束）之间划分工作，以减少通信和共享内存的读/写操作。

在第 4 节中，我们通过实证验证了 FlashAttention-2 相比于 FlashAttention 具有显著的加速比。在不同设置（有无因果掩码、不同注意力头数）下的基准测试显示，FlashAttention-2 相较于 FlashAttention 实现约 2 $\times$ 加速比，在前向传播中最高可达到理论最大吞吐量的 73%，在反向传播中最高可达到理论最大吞吐量的 63%。当端到端用于训练 GPT 风格模型时，我们每块 A100 GPU 的训练速度最高可达 225 TFLOPs/s。

2 背景

我们提供一些关于 GPU 的性能特征和执行模型的背景。我们还描述标准的注意实现，以及 FlashAttention。

2.1 硬件特征

GPU 性能特性。 GPU 由计算单元（例如浮点运算单元）和内存层次结构组成。大多数现代 GPU 包含用于加速低精度矩阵乘法的专用单元（例如，英伟达 GPU 上用于 FP16/BF16 矩阵乘法的 Tensor Cores）。内存层次结构包括高带宽内存（HBM）以及片上 SRAM（亦称共享内存）。例如，A100 GPU 具有 40-80GB 的高带宽内存（HBM），带宽为 1.5-2.0TB/s，并且在 108 个流式多处理器的每一个上都配有 192KB 的片上 SRAM，其带宽估计约为 19TB/s [6, 7]。由于程序员无法直接控制 L2 缓存，为便于讨论，我们将重点放在 HBM 和 SRAM 上。

执行模型。 GPU 拥有海量线程来执行一个操作（称为 kernel）。线程被组织为线程块，这些线程块会在流式多处理器（SM）上运行。在线程块内，线程被分组为 warp（束）（由 32 个线程组成）。同一线程束内的线程可以通过快速 shuffle 指令进行通信，或协同执行矩阵乘法。一个线程块内的 warp（束）可以通过从 / 向共享内存读取 / 执行写操作来通信。每个 kernel 会将输入从 HBM 加载到寄存器和 SRAM，进行计算，然后对 HBM 执行输出的写操作。

2.2 StandardAttention 实现

给定输入序列 $\mathbf{Q}, \mathbf{K}, \mathbf{V} \in \mathbb{R}^{N \times d}$ 其中 N 为序列长度, d 为头维度, 我们希望计算由 注意 产生的 输出 $\mathbf{O} \in \mathbb{R}^{N \times d}$:

$$\mathbf{S} = \mathbf{Q}\mathbf{K}^\top \in \mathbb{R}^{N \times N}, \quad \mathbf{P} = \text{softmax}(\mathbf{S}) \in \mathbb{R}^{N \times N}, \quad \mathbf{O} = \mathbf{P}\mathbf{V} \in \mathbb{R}^{N \times d},$$

其中 Softmax 按行应用。² 对于多头注意 (MHA), 同样的计算在多个头上并行执行, 并在批次维度 (一个批次中的输入序列数量) 上并行。

注意的反向传播如下进行。令 $d\mathbf{O} \in \mathbb{R}^{N \times d}$ 为 $\mathbf{O}$ 关于某个损失函数的梯度。于是根据链式法则 (又称反向传播):

$$\begin{aligned} d\mathbf{V} &= \mathbf{P}^\top d\mathbf{O} \in \mathbb{R}^{N \times d} \\ d\mathbf{P} &= d\mathbf{O}\mathbf{V}^\top \in \mathbb{R}^{N \times N} \\ d\mathbf{S} &= d\text{softmax}(d\mathbf{P}) \in \mathbb{R}^{N \times N} \\ d\mathbf{Q} &= d\mathbf{S}\mathbf{K} \in \mathbb{R}^{N \times d} \\ d\mathbf{K} &= \mathbf{Q}d\mathbf{S}^\top \in \mathbb{R}^{N \times d}, \end{aligned}$$

其中 dsoftmax 是按行应用的 Softmax 的梯度 (反向传播)。可以推导出, 如果 $p = \text{Softmax}(s)$ 对于某个向量 s 和 p , 则在具有输出梯度 dp 的情况下, 输入梯度为 $ds = (\text{diag}(p) - pp^\top)dp$ 。

标准注意的实现会将矩阵 $\mathbf{S}$ 和 $\mathbf{P}$ 写入 HBM, 这会占用 $O(N^2)$ 内存。通常 $N \gg d$ (通常 N 在 1k–8k 数量级, 而 d 大约在 64–128)。标准注意实现 (1) 调用 GEMM (通用矩阵乘法) 子程序以将 $\mathbf{S} = \mathbf{Q}\mathbf{K}^\top$ 相乘, 将结果写入 HBM, 然后 (2) 从 HBM 加载 $\mathbf{S}$ 以计算 Softmax, 并将结果 $\mathbf{P}$ 写入 HBM, 最后 (3) 调用 GEMM (通用矩阵乘法) 得到 $\mathbf{O} = \mathbf{P}\mathbf{V}$ 。由于大多数操作受内存带宽限制, 大量的内存访问会导致墙钟时间变慢。此外, 由于必须物化 $\mathbf{S}$ 和 $\mathbf{P}$, 所需内存为 $O(N^2)$ 。此外, 还必须保存 $\mathbf{P} \in \mathbb{R}^{N \times N}$ 以用于反向传播计算梯度。

2.3 FlashAttention

为了在诸如 GPU 等硬件加速器上加速 注意, [5] 提出了一种算法, 在保持相同输出 (无近似) 的前提下减少内存读/写操作。

2.3.1 前向传播

FlashAttention 采用经典的分块技术来减少内存 I/O: (1) 将输入块从 HBM 加载到 SRAM, (2) 针对该块计算注意, 随后 (3) 在无需对大型中间表示矩阵 $\mathbf{S}$ 和 $\mathbf{P}$ 向 HBM 执行写操作的情况下更新输出。由于 Softmax 会耦合整行或成块的行, online Softmax [11, 13] 可以将注意计算划分为块, 并对每个块的输出重新缩放, 从而最终得到正确结果 (无近似)。通过显著减少内存读/写操作, FlashAttention 相较于优化过的基线注意实现, 带来 2–4× 倍挂钟时间加速比。

我们介绍在线 Softmax 技术 [11] 以及它如何用于 注意 [13]。为简单起见, 只考虑 注意 矩阵 $\mathbf{S}$ 的一个行块, 其形式为 $[\mathbf{S}^{(1)}\mathbf{S}^{(2)}]$ 对应某些矩阵 $\mathbf{S}^{(1)}, \mathbf{S}^{(2)} \in \mathbb{R}^{B_r \times B_c}$, 其中 B_r 和 B_c 是行和列块的大小。我们希望对该行块计算 Softmax, 并与 数值 相乘, 其形式为 $\begin{bmatrix} \mathbf{V}^{(1)} \\ \mathbf{V}^{(2)} \end{bmatrix} \mathbf{V}$ 对应某些矩阵 $\mathbf{V}^{(1)}, \mathbf{V}^{(2)} \in \mathbb{R}^{B_c \times d}$ 。标准的 Softmax 将会

²为便于阐述, 我们省略 $\mathbf{Q}\mathbf{K}^\top$ 的缩放 (通常为 $1/d$), 以及对 $\mathbf{S}$ 进行逐元素掩码和/或对 $\mathbf{P}$ 应用 Dropout (随机失活) 的可选操作

计算:

$$\begin{aligned}
m &= \max(\text{rowmax}(\mathbf{S}^{(1)}), \text{rowmax}(\mathbf{S}^{(2)})) \in \mathbb{R}^{B_r} \\
\ell &= \text{rowsum}(e^{\mathbf{S}^{(1)}-m}) + \text{rowsum}(e^{\mathbf{S}^{(2)}-m}) \in \mathbb{R}^{B_r} \\
\mathbf{P} &= [\mathbf{P}^{(1)} \quad \mathbf{P}^{(2)}] = \text{diag}(\ell)^{-1} \begin{bmatrix} e^{\mathbf{S}^{(1)}-m} & e^{\mathbf{S}^{(2)}-m} \end{bmatrix} \in \mathbb{R}^{B_r \times 2B_c} \\
\mathbf{O} &= [\mathbf{P}^{(1)} \quad \mathbf{P}^{(2)}] \begin{bmatrix} \mathbf{V}^{(1)} \\ \mathbf{V}^{(2)} \end{bmatrix} = \text{diag}(\ell)^{-1} e^{\mathbf{S}^{(1)}-m} \mathbf{V}^{(1)} + e^{\mathbf{S}^{(2)}-m} \mathbf{V}^{(2)} \in \mathbb{R}^{B_r \times d}.
\end{aligned}$$

相较之下, Online Softmax 会针对每个块计算“局部” Softmax, 并进行重新缩放, 以在结束时得到正确的输出:

$$\begin{aligned}
m^{(1)} &= \text{rowmax}(\mathbf{S}^{(1)}) \in \mathbb{R}^{B_r} \\
\ell^{(1)} &= \text{rowsum}(e^{\mathbf{S}^{(1)}-m^{(1)}}) \in \mathbb{R}^{B_r} \\
\tilde{\mathbf{P}}^{(1)} &= \text{diag}(\ell^{(1)})^{-1} e^{\mathbf{S}^{(1)}-m^{(1)}} \in \mathbb{R}^{B_r \times B_c} \\
\mathbf{O}^{(1)} &= \tilde{\mathbf{P}}^{(1)} \mathbf{V}^{(1)} = \text{diag}(\ell^{(1)})^{-1} e^{\mathbf{S}^{(1)}-m^{(1)}} \mathbf{V}^{(1)} \in \mathbb{R}^{B_r \times d} \\
m^{(2)} &= \max(m^{(1)}, \text{rowmax}(\mathbf{S}^{(2)})) = m \\
\ell^{(2)} &= e^{m^{(1)}-m^{(2)}} \ell^{(1)} + \text{rowsum}(e^{\mathbf{S}^{(2)}-m^{(2)}}) = \text{rowsum}(e^{\mathbf{S}^{(1)}-m}) + \text{rowsum}(e^{\mathbf{S}^{(2)}-m}) = \ell \\
\tilde{\mathbf{P}}^{(2)} &= \text{diag}(\ell^{(2)})^{-1} e^{\mathbf{S}^{(2)}-m^{(2)}} \\
\mathbf{O}^{(2)} &= \text{diag}(\ell^{(1)}/\ell^{(2)})^{-1} \mathbf{O}^{(1)} + \tilde{\mathbf{P}}^{(2)} \mathbf{V}^{(2)} = \text{diag}(\ell^{(2)})^{-1} e^{\mathbf{S}^{(1)}-m} \mathbf{V}^{(1)} + \text{diag}(\ell^{(2)})^{-1} e^{\mathbf{S}^{(2)}-m} \mathbf{V}^{(2)} = \mathbf{O}.
\end{aligned}$$

我们展示如何 FlashAttention 使用在线 Softmax 以实现分块 (图 1), 从而减少内存读/写操作。

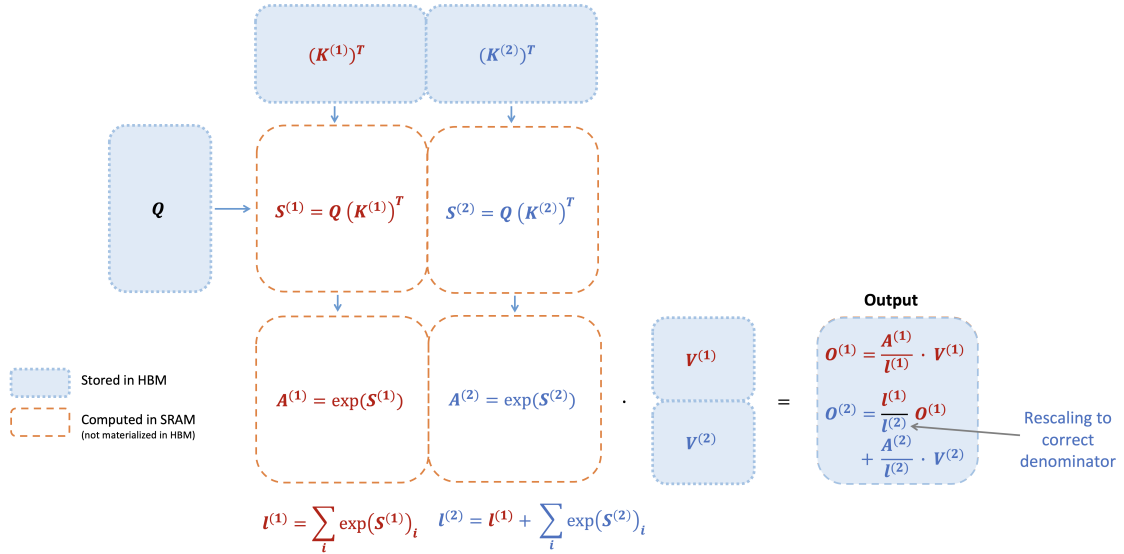


图1: 当键 $\mathbf{K}$ 被划分为两个块且数值 $\mathbf{V}$ 也被划分为两个块时, FlashAttention 前向传播的执行示意图。通过针对每个块计算注意并对输出进行重新缩放, 我们在结束时可以得到正确的结果, 同时避免对中间表示矩阵 $\mathbf{S}$ 和 $\mathbf{P}$ 的昂贵内存读/写操作。我们对该图进行了简化, 省略了 Softmax 中将每个元素减去按行最大值的步骤。

2.3.2 反向传播

在反向传播中，通过重新计算注意矩阵 $\mathbf{S}$ 和 $\mathbf{P}$ 的值，一旦输入块 $\mathbf{Q}, \mathbf{K}, \mathbf{V}$ 已加载到 SRAM, FlashAttention 避免了必须存储大型中间值。由于无需保存大小为 $N \times N$ 的大型矩阵 $\mathbf{S}$ 和 $\mathbf{P}$ ，FlashAttention 可根据序列长度实现 10-20 $\times$ 的内存节省（所需内存随序列长度 N 线性增长，而非二次增长）。由于减少内存读/写操作，反向传播还实现了 2-4 $\times$ 的挂钟时间加速比。

反向传播对第2.2节中的方程应用了分块。尽管从概念上说，反向传播比前向传播更简单（没有 Softmax 重新缩放），但实现要复杂得多。这是因为在反向传播中需要在 SRAM 中保留更多的值以执行 5 次矩阵乘法，而前向传播中仅需 2 次矩阵乘法。

3 FlashAttention-2: 算法、并行和工作划分

我们介绍 FlashAttention-2 算法，其中包含对 FlashAttention 的若干调整，以减少非 matmul FLOPs 的数量。接着，我们说明如何在不同线程块上并行这些计算，以充分利用 GPU 资源。最后，我们说明如何在单个线程块内在不同 warp（束）之间划分工作，以减少共享内存访问量。这些改进带来了 2-3 $\times$ 加速比，这一点在第4节中得到验证。

3.1 算法

我们对 FlashAttention 的算法做了调整，以减少非 matmul FLOPs 的数量。这是因为现代 GPU 具有专用计算单元（例如英伟达 GPU 上的 Tensor Cores），使得 matmul 要快得多。以 A100 GPU 为例，其 FP16/BF16 matmul 的最大理论吞吐量为 312 TFLOPs/s，而非 matmul 的 FP32 仅为 19.5 TFLOPs/s。换一种说法，每个非 matmul FLOP 比一个 matmul FLOP 16 $\times$ 更昂贵。为保持高吞吐量（例如达到最大理论 TFLOPs/s 的 50% 以上），我们希望尽可能将时间花在 matmul FLOPs 上。

3.1.1 前向传播

我们重温第 2.3 节所示的在线 Softmax 技巧，并做出两项小改动以减少非矩阵乘法的 FLOPs：

1. 我们不必用 $\text{diag}(\ell^{(2)})^{-1}$ 对输出更新的两项重新缩放：

$$\mathbf{O}^{(2)} = \text{diag}(\ell^{(1)}/\ell^{(2)})^{-1} \mathbf{O}^{(1)} + \text{diag}(\ell^{(2)})^{-1} e^{\mathbf{S}^{(2)} - \mathbf{m}^{(2)}} \mathbf{V}^{(2)}.$$

我们可以改为维护 $\mathbf{O}^{(2)}$ 的“未缩放”版本，并保留统计量 $\ell^{(2)}$ ：

$$\tilde{\mathbf{O}}^{(2)} = \text{diag}(\ell^{(1)})^{-1} \mathbf{O}^{(1)} + e^{\mathbf{S}^{(2)} - \mathbf{m}^{(2)}} \mathbf{V}^{(2)}.$$

仅在循环的最后，我们才将最终 $\tilde{\mathbf{O}}^{(\text{last})}$ 用 $\text{diag}(\ell^{(\text{last})})^{-1}$ 进行缩放，以得到正确的输出 t.

2. 我们不必同时保存最大值 $m^{(j)}$ 和指数和 $\ell^{(j)}$ 以用于反向传播。我们只需要存储 $\log \text{sumexp } L^{(j)} = m^{(j)} + \log(\ell^{(j)})$.

在第 2.3 节的 2 个块的简单情形下，在线 Softmax 技巧现在变为：

$$\begin{aligned}
m^{(1)} &= \text{rowmax}(\mathbf{S}^{(1)}) \in \mathbb{R}^{B_r} \\
\ell^{(1)} &= \text{rowsum}(e^{\mathbf{S}^{(1)} - m^{(1)}}) \in \mathbb{R}^{B_r} \\
\tilde{\mathbf{O}}^{(1)} &= e^{\mathbf{S}^{(1)} - m^{(1)}} \mathbf{V}^{(1)} \in \mathbb{R}^{B_r \times d} \\
m^{(2)} &= \max(m^{(1)}, \text{rowmax}(\mathbf{S}^{(2)})) = m \\
\ell^{(2)} &= e^{m^{(1)} - m^{(2)}} \ell^{(1)} + \text{rowsum}(e^{\mathbf{S}^{(2)} - m^{(2)}}) = \text{rowsum}(e^{\mathbf{S}^{(1)} - m} + e^{\mathbf{S}^{(2)} - m}) = \ell \\
\tilde{\mathbf{P}}^{(2)} &= \text{diag}(\ell^{(2)})^{-1} e^{\mathbf{S}^{(2)} - m^{(2)}} \\
\tilde{\mathbf{O}}^{(2)} &= \text{diag}(e^{m^{(1)} - m^{(2)}})^{-1} \tilde{\mathbf{O}}^{(1)} + e^{\mathbf{S}^{(2)} - m^{(2)}} \mathbf{V}^{(2)} = e^{s^{(1)} - m} \mathbf{V}^{(1)} + e^{s^{(2)} - m} \mathbf{V}^{(2)} \\
\mathbf{O}^{(2)} &= \text{diag}(\ell^{(2)})^{-1} \tilde{\mathbf{O}}^{(2)} = \mathbf{O}.
\end{aligned}$$

我们在算法 1 中描述完整的 FlashAttention-2 前向传播。

Algorithm 1 FlashAttention-2 前向传播

要求：矩阵 $\mathbf{Q}, \mathbf{K}, \mathbf{V} \in \mathbb{R}^{N \times d}$ 在 HBM 中，块大小为 B_c, B_r 。1: 将 $\mathbf{Q}$ 划分为 $T_r = \left\lceil \frac{N}{B_r} \right\rceil$ 个块 $\mathbf{Q}_1, \dots, \mathbf{Q}_{T_r}$ ，每个大小为 $B_r \times d$ ；并将 $\mathbf{K}, \mathbf{V}$ 划分为 $T_c = \left\lceil \frac{N}{B_c} \right\rceil$ 个块 $\mathbf{K}_1, \dots, \mathbf{K}_{T_c}$ 和 $\mathbf{V}_1, \dots, \mathbf{V}_{T_c}$ ，每个大小为 $B_c \times d$ 。2: 将输出 $\mathbf{O} \in \mathbb{R}^{N \times d}$ 划分为 T_r 个块 $\mathbf{O}_i, \dots, \mathbf{O}_{T_r}$ ，每个大小为 $B_r \times d$ ；并将 $\text{logsumexp } L$ 划分为 T_r 个块 $L_i, \dots, L_{T_r}$ ，每个大小为 B_r 。3: for $1 \leq i \leq T_r$ do 4: 将 $\mathbf{Q}_i$ 从 HBM 加载到片上 SRAM。5: 在片上，初始化 $\mathbf{O}_i^{(0)} = (0)_{B_r \times d} \in \mathbb{R}^{B_r \times d}, \mathbf{l}_i^{(0)} = (0)_{B_r} \in \mathbb{R}^{B_r}, m_i^{(0)} = (-\infty)_{B_r} \in \mathbb{R}^{B_r}$ 。6: for $1 \leq j \leq T_c$ do 7: 将 $\mathbf{K}_j, \mathbf{V}_j$ 从 HBM 加载到片上 SRAM。8: 在片上，计算 $\mathbf{S}^{(j)}_i = \mathbf{Q}_i \mathbf{K}_j^T \in \mathbb{R}^{B_r \times B_c}$ 。9: 在片上，计算 $m^{(j)}_i = \max(m^{(j-1)}_i, \text{rowmax}(\mathbf{S}^{(j)}_i)) \in \mathbb{R}^{B_r}, \tilde{\mathbf{P}}^{(j)}_i = \exp(\mathbf{S}^{(j)}_i - m^{(j)}_i) \in \mathbb{R}^{B_r \times B_c}$ (逐点), $\mathbf{l}^{(j)}_i = e^{m^{(j-1)}_i - m^{(j)}_i} \mathbf{l}^{(j-1)}_i + \text{rowsum}(\tilde{\mathbf{P}}^{(j)}_i) \in \mathbb{R}^{B_r}$ 。10: 在片上，计算 $\mathbf{O}^{(j)}_i = \text{diag}(e^{m^{(j-1)}_i - m^{(j)}_i})^{-1} \mathbf{O}^{(j-1)}_i + \tilde{\mathbf{P}}^{(j)}_i \mathbf{V}_j$ 。11: 结束 for 12: 在片上，计算 $\mathbf{O}_i = \text{diag}(\mathbf{l}^{(T_c)}_i)^{-1} \mathbf{O}^{(T_c)}_i$ 。13: 在片上，计算 $L_i = m^{(T_c)}_i + \log(\mathbf{l}^{(T_c)}_i)$ 。14: 写操作：将 $\mathbf{O}_i$ 写入 HBM，作为 $\mathbf{O}$ 的第 i 个块。15: 写操作：将 L_i 写入 HBM，作为 L 的第 i 个块。16: 结束 for 17: 返回输出 $\mathbf{O}$ 和 $\text{logsumexp } L$ 。

因果掩码。 注意的一种常见用例是在自回归语言建模中，我们需要对注意矩阵 $\mathbf{S}$ 应用因果掩码（即，任何满足 $j > i$ 的条目 $\mathbf{S}_{ij}$ 被设为 $-\infty$ ）。

1. 由于 FlashAttention 和 FlashAttention-2 已经按块进行，对于任意块，如果该块中所有列索引都大于行索引（当序列长度较大时约占一半的块），我们可以跳过该块的计算。这样相比于不使用因果掩码的注意，可带来约 1.7-1.8 $\times$ 加速比。2. 对于那些行索引保证严格小于列索引的块，我们无需应用因果掩码。这意味着对于每一行，我们只需对 1 个块应用因果掩码（假设方形块）。

正确性、运行时间与内存需求。 与 FlashAttention 一样，算法 1 返回正确的输出 $\mathbf{O} = \text{softmax}(\mathbf{QK}^\top)\mathbf{V}$ （无近似），使用 $O(N^2d)$ FLOPs，并且在输入和输出之外还需要 $O(N)$ 额外内存（用于存储 $\text{logsumexp } L$ ）。该证明与 Dao 等人 [5, 定理 1] 的证明几乎相同，因此我们在此略去。

3.1.2 反向传播

FlashAttention-2 的反向传播与 FlashAttention 的几乎相同。我们做了一个小改动，仅使用按行的 $\text{logsumexp } L$ 而不是在 Softmax 中同时使用按行最大值和按行指数和。为完整起见，我们在算法2中给出反向传播的描述。

算法2 FlashAttention-2 反向传播 要求：矩阵 $\mathbf{Q}$ 、 $\mathbf{K}$ 、 $\mathbf{V}$ 、 $\mathbf{O}$ （输出矩阵）、 $\mathbf{dO} \in \mathbb{R}^{N \times d}$ 在 HBM 中，向量 $\mathbf{L} \in \mathbb{R}^N$ 在 HBM 中，块大小 B_c, B_r 。1: 将 $\mathbf{Q}$ 划分为 $T_r = \left\lceil \frac{N}{B_r} \right\rceil$ 个块 $\mathbf{Q}_1, \dots, \mathbf{Q}_{T_r}$ ，每个大小为 $B_r \times d$ ，并将 $\mathbf{K}$ 、 $\mathbf{V}$ 划分为 $T_c = \left\lceil \frac{N}{B_c} \right\rceil$ 个块 $\mathbf{K}_1, \dots, \mathbf{K}_{T_c}$ 和 $\mathbf{V}_1, \dots, \mathbf{V}_{T_c}$ ，每个大小为 $B_c \times d$ 。2: 将 $\mathbf{O}$ 划分为 T_r 个块 $\mathbf{O}_1, \dots, \mathbf{O}_{T_r}$ ，每个大小为 $B_r \times d$ ，将 $\mathbf{dO}$ 划分为 T_r 个块 $\mathbf{dO}_1, \dots, \mathbf{dO}_{T_r}$ ，每个大小为 $B_r \times d$ ，并将 $\mathbf{L}$ 划分为 T_r 个块 $L_1, \dots, L_{T_r}$ ，每个大小为 B_r 。3: 初始化 $\mathbf{dQ} = (0)_{N \times d}$ 在 HBM 中，并将其划分为 T_r 个块 $\mathbf{dQ}_1, \dots, \mathbf{dQ}_{T_r}$ ，每个大小为 $B_r \times d$ 。将 $\mathbf{dK}$ 、 $\mathbf{dV} \in \mathbb{R}^{N \times d}$ 划分为 T_c 个块 $\mathbf{dK}_1, \dots, \mathbf{dK}_{T_c}$ 和 $\mathbf{dV}_1, \dots, \mathbf{dV}_{T_c}$ ，每个大小为 $B_c \times d$ 。4: 计算 $\mathbf{D} = \text{rowsum}(\mathbf{dO} \circ \mathbf{O}) \in \mathbb{R}^d$ (逐点相乘)，将 $\mathbf{D}$ 写入 HBM 并将其划分为 T_r 个块 $\mathbf{D}_1, \dots, \mathbf{D}_{T_r}$ ，每个大小为 B_r 。5: for $1 \leq j \leq T_c$ do 6: 将 $\mathbf{K}_j$ 、 $\mathbf{V}_j$ 从 HBM 加载到片上 SRAM。7: 在 SRAM 上初始化 $\mathbf{dK}_j = (0)_{B_c \times d}$ 、 $\mathbf{dV}_j = (0)_{B_c \times d}$ 。8: for $1 \leq i \leq T_r$ do 9: 将 $\mathbf{Q}_i$ 、 $\mathbf{O}_i$ 、 $\mathbf{dO}_i$ 、 $\mathbf{dQ}_i$ 、 L_i 、 $\mathbf{D}_i$ 从 HBM 加载到片上 SRAM。10: 在片上，计算 $\mathbf{S}^{(j)}_i = \mathbf{Q}_i \mathbf{K}_j^\top \in \mathbb{R}^{B_r \times B_c}$ 。11: 在片上，计算 $\mathbf{P}^{(j)}_i = \exp(\mathbf{S}_{ij} - L_i) \in \mathbb{R}^{B_r \times B_c}$ 。12: 在片上，计算 $\mathbf{dV}_j \leftarrow \mathbf{dV}_j + (\mathbf{P}^{(j)}_i)^\top \mathbf{dO}_i \in \mathbb{R}^{B_c \times d}$ 。13: 在片上，计算 $\mathbf{dP}^{(j)}_i = \mathbf{dO}_i \mathbf{V}_j^\top \in \mathbb{R}^{B_r \times B_c}$ 。14: 在片上，计算 $\mathbf{dS}^{(j)}_i = \mathbf{P}^{(j)}_i \circ (\mathbf{dP}^{(j)}_i - \mathbf{D}_i) \in \mathbb{R}^{B_r \times B_c}$ 。15: 将 $\mathbf{dQ}_i$ 从 HBM 加载到 SRAM，然后在片上，更新 $\mathbf{dQ}_i \leftarrow \mathbf{dQ}_i + \mathbf{dS}^{(j)}_i \mathbf{K}_j \in \mathbb{R}^{B_r \times d}$ ，并回写到 HBM。16: 在片上，计算 $\mathbf{dK}_j \leftarrow \mathbf{dK}_j + \mathbf{dS}^{(j)}_i{}^\top \mathbf{Q}_i \in \mathbb{R}^{B_c \times d}$ 。17: end for 18: 将 $\mathbf{dK}_j$ 、 $\mathbf{dV}_j$ 写入 HBM。19: end for 20: 返回 $\mathbf{dQ}$ 、 $\mathbf{dK}$ 、 $\mathbf{dV}$ 。

多查询注意与分组查询注意。 多查询注意 (MQA) [15] 与分组查询注意 (GQA) [1] 是注意的变体，其中多个查询头关注同一个键与数值头，以在推理期间减少 KV 缓存的大小。我们无需为计算复制键与数值头，而是通过对头的索引进行隐式操作来执行相同的计算。在反向传播中，我们需要在那些被隐式复用的不同头之间对梯度 $\mathbf{dK}$ 和 $\mathbf{dV}$ 求和。

3.2 并行

FlashAttention 的第一版在批大小和注意力头数上并行化。我们使用 1 个线程块来处理一个注意力头，总共有 批大小 $\cdot$ 注意力头数 个线程块。每个线程块被调度在一个流式多处理器 (SM) 上运行，并且有 108 个这样的 SM 在

一块 A100 GPU 上，例如。这种调度在该数值较大（例如 ≥ 80 ）时是高效的，因为我们可以有效地利用 GPU 上几乎所有的计算资源。

在长序列的情况下（这通常意味着较小的批大小或较少的注意力头数），为了更好地利用 GPU 上的多处理器，我们现在还在序列长度维度上进行并行化。这在这种情形下带来了显著的加速比。

前向传播。 我们发现外层循环（在序列长度上）是天然并行的，我们将它们调度到彼此无需通信的不同线程块上。我们也像在 FlashAttention 中那样，沿批大小维度和注意力头数维度进行并行化。当批大小和注意力头数较小时，在序列长度上的更高并行度有助于提高占用率（已使用的 GPU 资源占比），从而在这种情况下带来加速比。

交换循环顺序（外层循环遍历行块、内层循环遍历列块，而不是原始 FlashAttention 论文中的相反方式），以及在序列长度维度上进行并行化的这些想法，最初由 Phil Tillet 在 Triton [17]实现中提出并实现。

3

反向传播。 请注意，不同列块之间唯一的共享计算出现在更新 dQ 在算法2中的步骤，在该处我们需要将 dQ_i 从 HBM 加载到 SRAM，然后在片上，更新 $dQ_i \leftarrow dQ_i + dS^{(j)}_i K_j$ ，并执行写操作回到 HBM。因此我们也在序列长度维度上并行化，并为反向传播的每个列块调度 1 个线程块。我们使用原子加法在不同线程块之间进行通信以更新 dQ 。

我们在图2中描述了并行化方案。

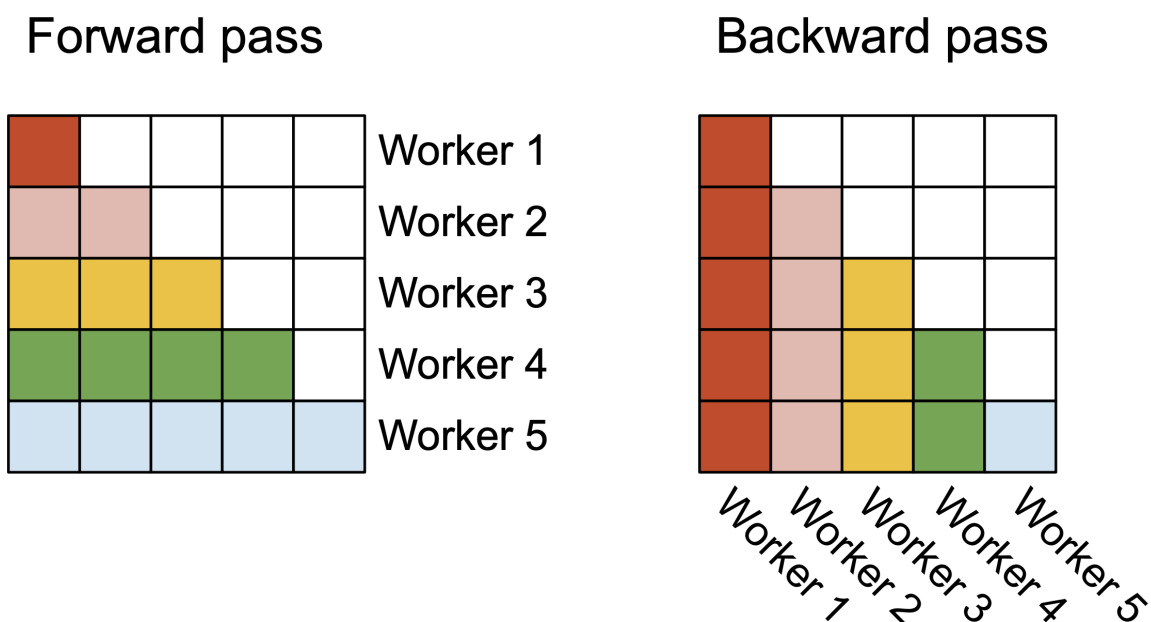


图2：在前向传播（左）中，我们并行化工作者（线程块），使每个工作者负责注意矩阵的一个行块。在反向传播（右）中，每个工作者负责注意矩阵的一个列块。

³<https://github.com/openai/triton/blob/main/python/tutorials/06-fused-attention.py>

3.3 warp（束）之间的工作划分

第3.2节介绍了我们如何调度线程块；在每个线程块内部，我们还需要决定如何在不同的 warp（束）之间划分工作。我们通常每个线程块使用4或8个 warp（束），其划分方式如图3所述。

前向传播。 对于每个块，FlashAttention 将 $\mathbf{K}$ 和 $\mathbf{V}$ 划分到4个 warp（束）上，同时让 $\mathbf{Q}$ 对所有 warp（束）可访问。每个线程束进行乘法以得到 $\mathbf{QK}^T$ 的一部分，然后需要与 $\mathbf{V}$ 的一部分相乘并进行通信以将结果相加。这被称为“split-K”方案。然而，这种方式效率不高，因为所有 warp（束）都需要将其中间表示写操作到共享内存、进行同步，然后再把中间表示相加。这些对共享内存的读/写操作会拖慢 FlashAttention 中的前向传播。

在 FlashAttention-2 中，我们改为将 $\mathbf{Q}$ 划分到4个 warp（束）上，同时让 $\mathbf{K}$ 和 $\mathbf{V}$ 对所有 warp（束）可访问。每个线程束完成矩阵乘法以得到 $\mathbf{QK}^T$ 的一部分后，只需与其共享的 $\mathbf{V}$ 切片相乘即可得到其对应的输出切片。warp（束）之间无需通信。减少对共享内存的读/写操作带来了加速比（第4节）。

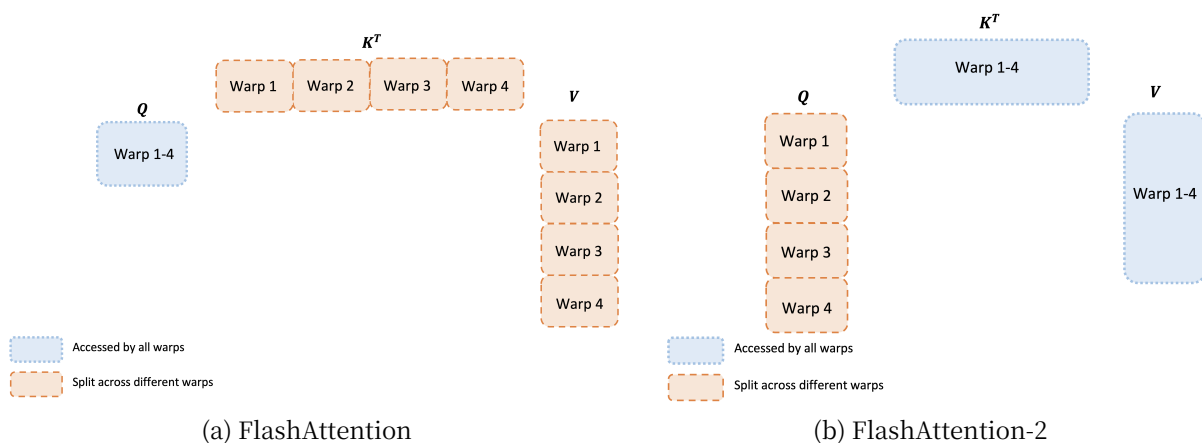


图3：前向传播中不同 warp（束）之间的工作划分

反向传播。 类似地，对于反向传播，我们选择对 warp（束）进行划分，以避免采用“split-K”方案。然而，由于所有不同的输入和梯度 $\mathbf{Q}, \mathbf{K}, \mathbf{V}, \mathbf{O}, \mathbf{dO}, \mathbf{dQ}, \mathbf{dK}, \mathbf{dV}$ 之间更复杂的依赖关系，仍然需要一定的同步。尽管如此，避免“split-K”可以减少对共享内存的读取/写操作，并再次带来加速比（第4节）。

块大小调优 增大块大小通常会减少对共享内存的加载/存储，但会增加所需的寄存器数量以及共享内存总量。超过某个块大小后，寄存器溢出会导致显著的减速；或者所需的共享内存量超过 GPU 可用容量，内核将完全无法运行。通常我们选择大小为 $\{64, 128\} \times \{64, 128\}$ 的块，具体取决于头部维度 d 以及设备共享内存大小。

我们针对每个头的维度进行手动调参，因为块大小本质上只有4种选择，但若使用自动调优，可以避免这些手动工作。我们将此留待未来工作。

4 实证验证

我们评估使用 FlashAttention-2 训练 Transformer 模型的影响。

- **对注意进行基准测试。** 我们测量 FlashAttention-2 在不同序列长度下的运行时间，并将其与 PyTorch 中的标准实现、FlashAttention，以及在 Triton 中的 FlashAttention 进行比较。我们确认 FlashAttention-2 比 FlashAttention 快 1.7–3.0× 倍，比在 Triton 中的 FlashAttention 快 1.3–2.5× 倍，并且比标准的注意实现快 3–10× 倍。

FlashAttention-2 最高可达 230 TFLOPs/s，为 A100 GPU 上理论最大 TFLOPs/s 的 73%。

- **端到端训练速度**当用于对序列长度为 2k 或 8k、大小为 13 亿和 27 亿的 GPT 风格模型进行端到端训练时，FlashAttention-2 相较于 FlashAttention 最高可获得 1.3× 的加速比，相较于不使用 FlashAttention 的基线最高可获得 2.8× 的加速比。FlashAttention-2 在每块 A100 GPU 上可达到最高 225 TFLOPs/s（模型 FLOPs 利用率为 72%）。

4.1 注意的基准测试

我们在 A100 80GB SXM4 GPU 上测量不同注意方法在不同设置下的运行时间（无/有因果掩码，头维度为 64 或 128）。我们在图 4、图 5 和图 6 中报告结果，显示 FlashAttention-2 大约比 FlashAttention 和 xformers 中的 FlashAttention（“cutlass”实现）快 2×。在前向传播中，FlashAttention-2 比 Triton 中的 FlashAttention 快约 1.3-1.5×，在反向传播中快约 2×。与 PyTorch 中的标准注意实现相比，FlashAttention-2 最多可快 10×。

基准设置：我们将序列长度从 512、1k、...、16k 进行变化，并设置批大小，使总令牌数量为 16k。我们将隐藏维度设为 2048，将头维度设为 64 或 128（即 32 个头或 16 个头）。为计算前向传播的 FLOPs，我们使用：

$$4 \cdot \text{seq len}^2 \cdot \text{head dimension} \cdot \text{number of heads}.$$

在使用因果掩码时，我们将该数除以 2，因为大约只有一半的条目会被计算。要得到反向传播的 FLOPs，我们将前向传播的 FLOPs 乘以 2.5（由于重计算，前向传播中有 2 次矩阵乘法，反向传播中有 5 次矩阵乘法）。

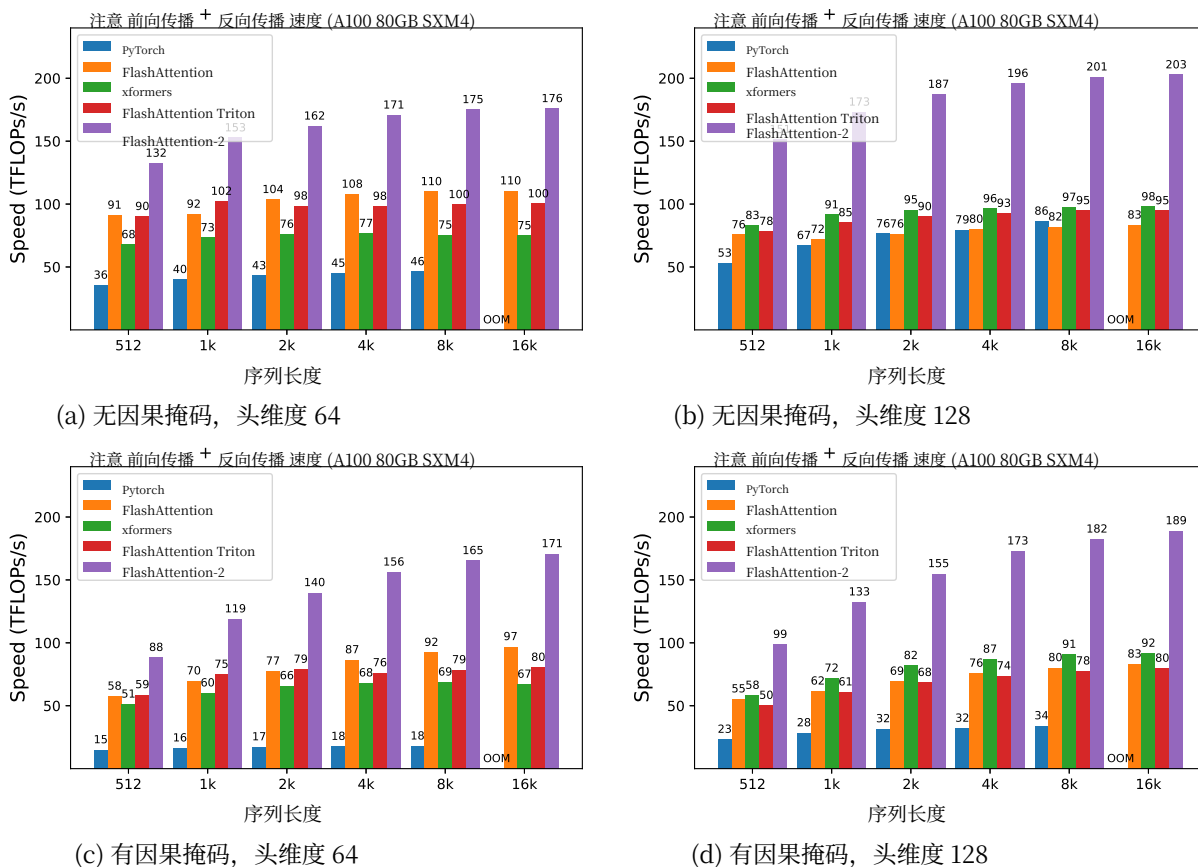
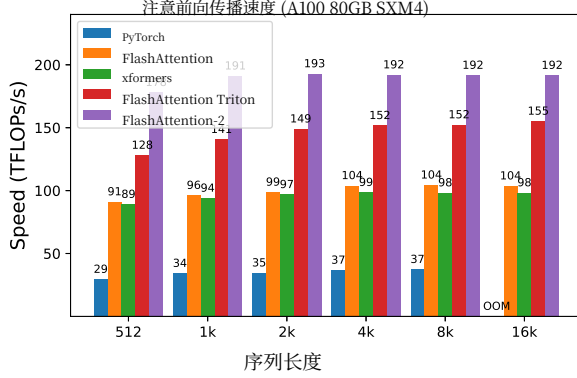
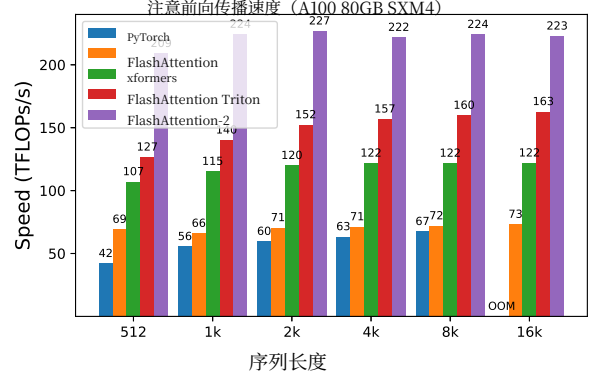


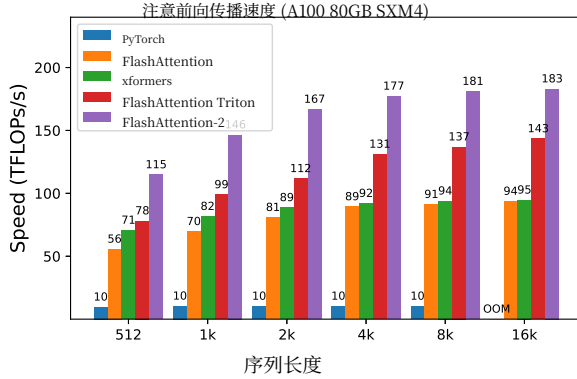
图4：注意 前向传播 + 反向传播 速度 在 A100 GPU 上



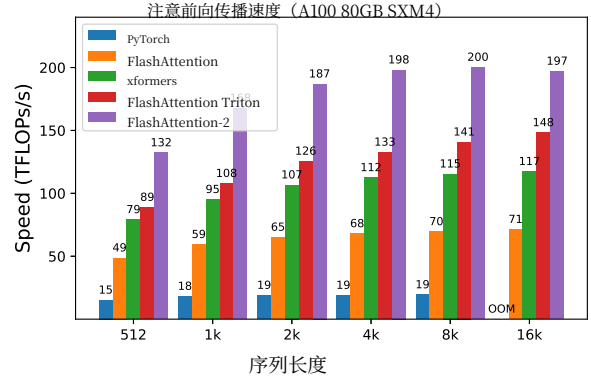
(a) 无因果掩码，头维度 64



(b) 无因果掩码，头维度 128



(c) 带因果掩码，头维度 64



(d) 带因果掩码，头维度 128

图5: A100 GPU 上的注意前向传播速度

仅在 H100 GPU 上运行相同的实现（未使用任何专门指令来利用 TMA 和第四代 Tensor Cores 等新特性），我们可获得最高 335 TFLOPs/s（图 7）。我们预计，通过使用新指令，在 H100 GPU 上还能获得额外 1.5 倍到 2 倍的加速比。我们将其留待未来工作。

4.2 端到端性能

我们在 8×A100 80GB SXM 上测量了具有 1.3B 或 2.7B 参数的 GPT 风格模型的训练吞吐量。如表1所示，FlashAttention-2 相较于不使用 FlashAttention 的基线带来 2.8× 加速比，并且相较于 FlashAttention-2 达到 1.3× 加速比，每张 A100 GPU 最高达到 225 TFLOPs/s。

请注意，我们按照 Megatron-LM [16]（以及许多其他论文和库）中的公式来计算 FLOPs：

$$6 \cdot \text{seq_len} \cdot \text{number of params} + 12 \cdot \text{number of layers} \cdot \text{hidden dim} \cdot \text{seq_len}^2.$$

第一项对应由权重-输入乘法产生的 FLOPs，第二项对应由注意产生的 FLOPs。然而，有人可能会认为第二项应该减半，因为在使用因果掩码时，我们只需要在注意中计算大约一半的元素数量。为了一致性，我们选择遵循文献中的公式（不将注意的 FLOPs 除以 2）。

5 讨论和未来方向

FlashAttention-2 比 FlashAttention 2× 更快，这意味着我们可以在与此前训练 8k 上下文模型相同的成本下，训练具有 16k 上下文的模型。我们对这将如何

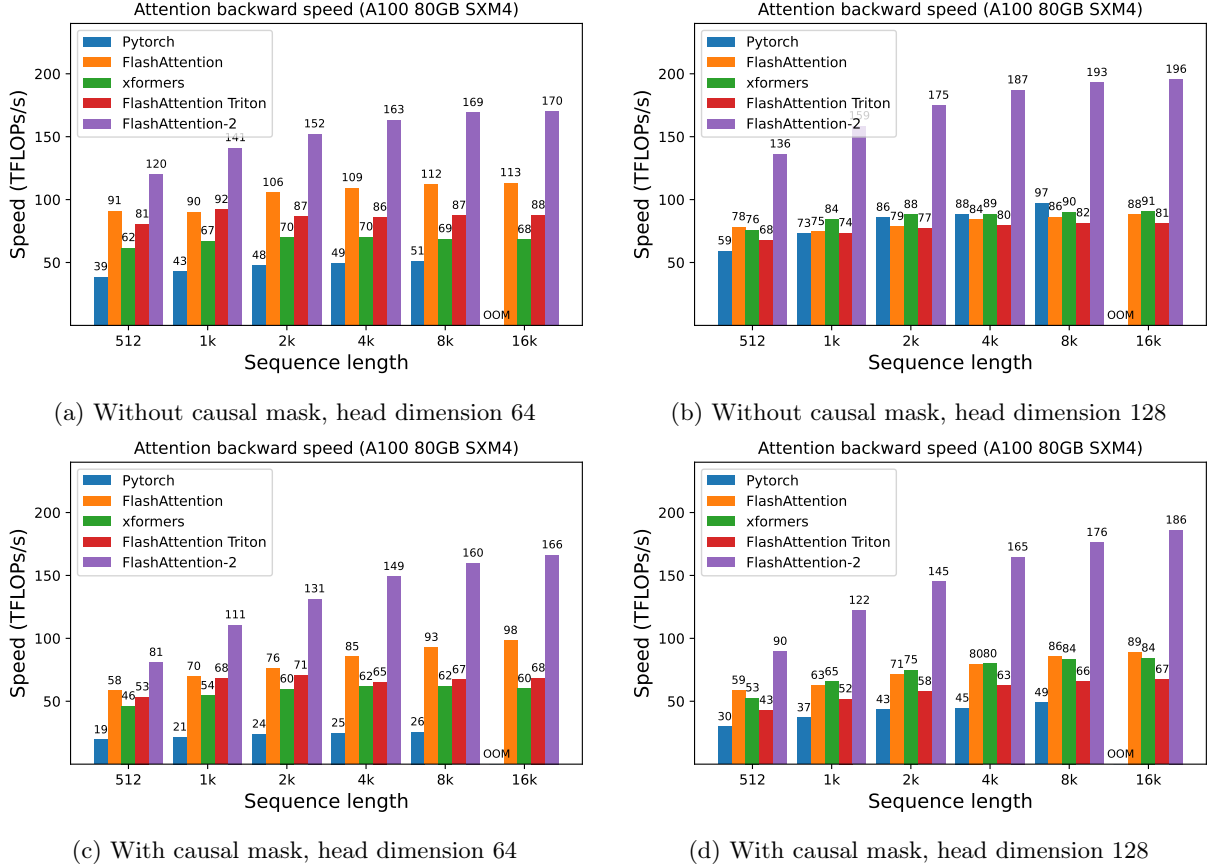


Figure 6: Attention backward speed on A100 GPU

Table 1: Training speed (TFLOPs/s/GPU) of GPT-style models on 8xA100 GPUs. FLASHATTENTION-2 reaches up to 225 TFLOPs/s (72% model FLOPs utilization). We compare against a baseline running without FLASHATTENTION.

Model	Without FLASHATTENTION	FLASHATTENTION	FLASHATTENTION-2
GPT3-1.3B 2k context	142 TFLOPs/s	189 TFLOPs/s	196 TFLOPs/s
GPT3-1.3B 8k context	72 TFLOPs/s	170 TFLOPs/s	220 TFLOPs/s
GPT3-2.7B 2k context	149 TFLOPs/s	189 TFLOPs/s	205 TFLOPs/s
GPT3-2.7B 8k context	80 TFLOPs/s	175 TFLOPs/s	225 TFLOPs/s

be used to understand long books and reports, high resolution images, audio and video. FLASHATTENTION-2 will also speed up training, finetuning, and inference of existing models.

In the near future, we plan to collaborate with researchers and engineers to make FlashAttention widely applicable in different kinds of devices (e.g., H100 GPUs, AMD GPUs), as well as new data types such as FP8. As an immediate next step, we plan to optimize FlashAttention-2 for H100 GPUs to use new hardware features (TMA, 4th-gen Tensor Cores, fp8). Combining the low-level optimizations in FlashAttention-2 with high-level algorithmic changes (e.g., local, dilated, block-sparse attention) could allow us to train AI models with much longer context. We are also excited to work with compiler researchers to make these optimization techniques easily programmable.

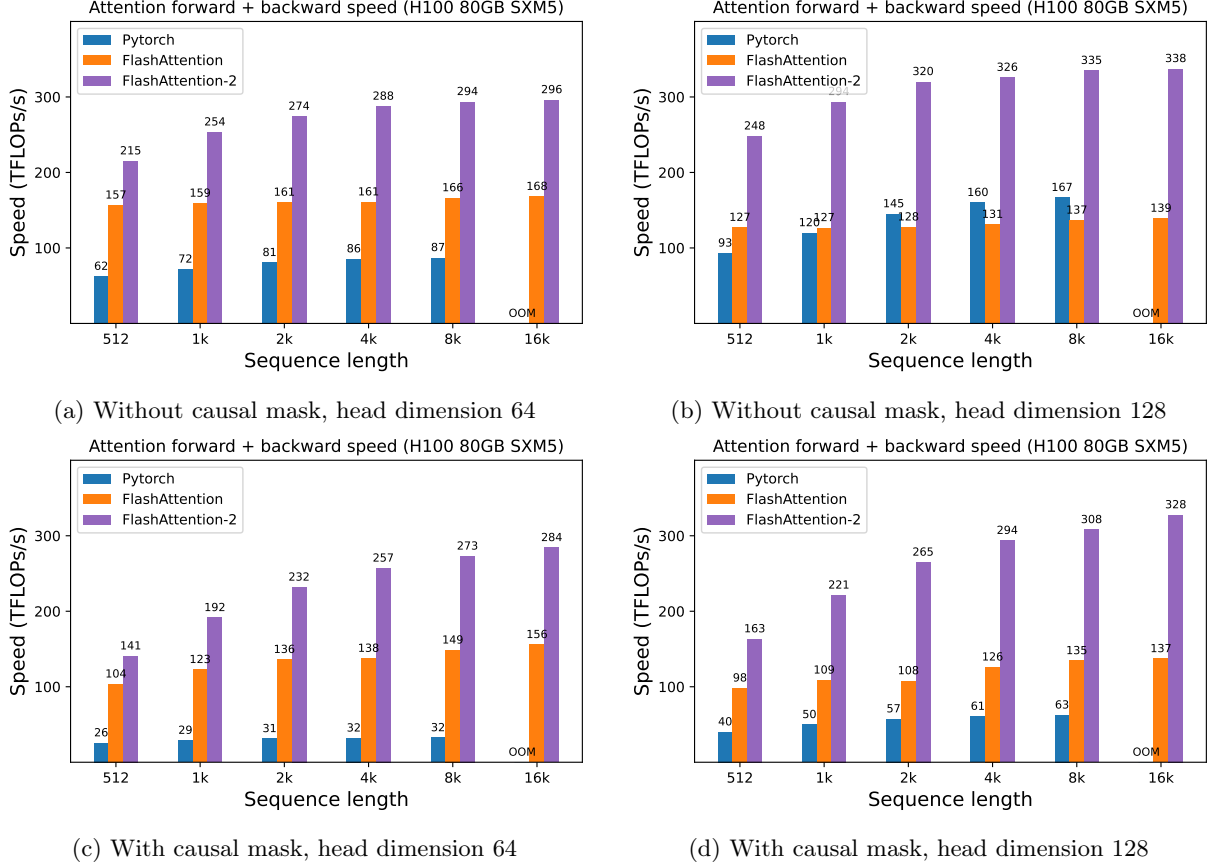


Figure 7: Attention forward + backward speed on H100 GPU

Acknowledgments

We thank Phil Tillet and Daniel Haziza, who have implemented versions of FLASHATTENTION in Triton [17] and the `xformers` library [10]. FLASHATTENTION-2 was motivated by exchange of ideas between different ways that attention could be implemented. We are grateful to the Nvidia CUTLASS team (especially Vijay Thakkar, Cris Cecka, Haicheng Wu, and Andrew Kerr) for their CUTLASS library, in particular the CUTLASS 3.x release, which provides clean abstractions and powerful building blocks for the implementation of FLASHATTENTION-2. We thank Driss Guessous for integrating FLASHATTENTION to PyTorch. FLASHATTENTION-2 has benefited from helpful discussions with Phil Wang, Markus Rabe, James Bradbury, Young-Jun Ko, Julien Launay, Daniel Hesslow, Michaël Benesty, Horace He, Ashish Vaswani, and Erich Elsen. Thanks for Stanford CRFM and Stanford NLP for the compute support. We thank Dan Fu and Christopher Ré for their collaboration, constructive feedback, and constant encouragement on this line of work of designing hardware-efficient algorithms. We thank Albert Gu and Beidi Chen for their helpful suggestions on early drafts of this technical report.

References

- [1] Joshua Ainslie, James Lee-Thorp, Michiel de Jong, Yury Zemlyanskiy, Federico Lebrón, and Sumit Sanghai. Gqa: Training generalized multi-query transformer models from multi-head checkpoints. *arXiv preprint arXiv:2305.13245*, 2023.
- [2] Iz Beltagy, Matthew E Peters, and Arman Cohan. Longformer: The long-document transformer. *arXiv preprint arXiv:2004.05150*, 2020.

- [3] Beidi Chen, Tri Dao, Eric Winsor, Zhao Song, Atri Rudra, and Christopher Ré. Scatterbrain: Unifying sparse and low-rank attention. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2021.
- [4] Krzysztof Marcin Choromanski, Valerii Likhoshesterov, David Dohan, Xingyou Song, Andreea Gane, Tamas Sarlos, Peter Hawkins, Jared Quincy Davis, Afroz Mohiuddin, Lukasz Kaiser, et al. Rethinking attention with performers. In *International Conference on Learning Representations (ICLR)*, 2020.
- [5] Tri Dao, Daniel Y. Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. FlashAttention: Fast and memory-efficient exact attention with IO-awareness. In *Advances in Neural Information Processing Systems*, 2022.
- [6] Zhe Jia and Peter Van Sandt. Dissecting the Ampere GPU architecture via microbenchmarking. GPU Technology Conference, 2021.
- [7] Zhe Jia, Marco Maggioni, Benjamin Staiger, and Daniele P Scarpazza. Dissecting the nvidia Volta GPU architecture via microbenchmarking. *arXiv preprint arXiv:1804.06826*, 2018.
- [8] Angelos Katharopoulos, Apoorv Vyas, Nikolaos Pappas, and François Fleuret. Transformers are RNNs: Fast autoregressive transformers with linear attention. In *International Conference on Machine Learning*, pages 5156–5165. PMLR, 2020.
- [9] Nikita Kitaev, Lukasz Kaiser, and Anselm Levskaya. Reformer: The efficient transformer. In *The International Conference on Machine Learning (ICML)*, 2020.
- [10] Benjamin Lefaudeux, Francisco Massa, Diana Liskovich, Wenhan Xiong, Vittorio Caggiano, Sean Naren, Min Xu, Jieru Hu, Marta Tintore, Susan Zhang, Patrick Labatut, and Daniel Haziza. xformers: A modular and hackable transformer modelling library. <https://github.com/facebookresearch/xformers>, 2022.
- [11] Maxim Milakov and Natalia Gimelshein. Online normalizer calculation for softmax. *arXiv preprint arXiv:1805.02867*, 2018.
- [12] OpenAI. Gpt-4 technical report. *ArXiv*, abs/2303.08774, 2023.
- [13] Markus N Rabe and Charles Staats. Self-attention does not need $O(n^2)$ memory. *arXiv preprint arXiv:2112.05682*, 2021.
- [14] Aurko Roy, Mohammad Saffar, Ashish Vaswani, and David Grangier. Efficient content-based sparse attention with routing transformers. *Transactions of the Association for Computational Linguistics*, 9: 53–68, 2021.
- [15] Noam Shazeer. Fast transformer decoding: One write-head is all you need. *arXiv preprint arXiv:1911.02150*, 2019.
- [16] Mohammad Shoeybi, Mostofa Patwary, Raul Puri, Patrick LeGresley, Jared Casper, and Bryan Catanzaro. Megatron-LM: Training multi-billion parameter language models using model parallelism. *arXiv preprint arXiv:1909.08053*, 2019.
- [17] Philippe Tillet, Hsiang-Tsung Kung, and David Cox. Triton: an intermediate language and compiler for tiled neural network computations. In *Proceedings of the 3rd ACM SIGPLAN International Workshop on Machine Learning and Programming Languages*, pages 10–19, 2019.
- [18] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. *Advances in neural information processing systems*, 30, 2017.
- [19] Sinong Wang, Belinda Z Li, Madian Khabsa, Han Fang, and Hao Ma. Linformer: Self-attention with linear complexity. *arXiv preprint arXiv:2006.04768*, 2020.
- [20] Manzil Zaheer, Guru Guruganesh, Kumar Avinava Dubey, Joshua Ainslie, Chris Alberti, Santiago Ontanon, Philip Pham, Anirudh Ravula, Qifan Wang, Li Yang, et al. Big bird: Transformers for longer sequences. *Advances in Neural Information Processing Systems*, 33, 2020.